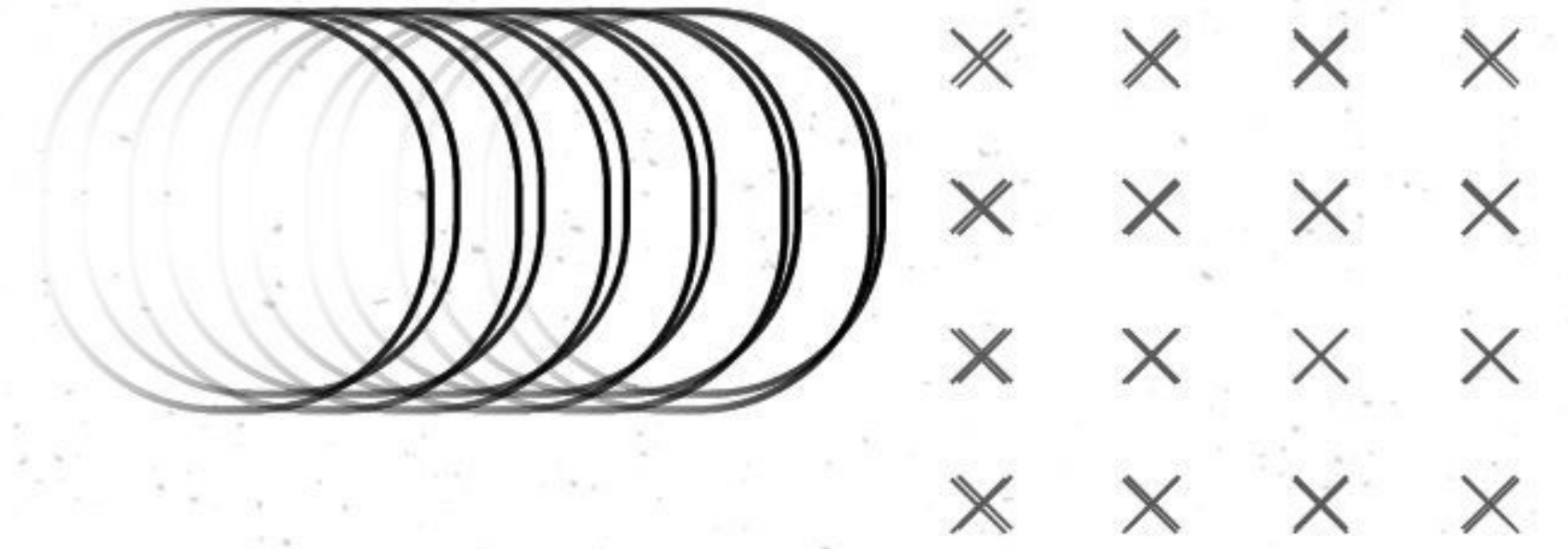




DATA STRUCTURE

LECTURE " 1 " : Introduction to Data
Structures & Complexity

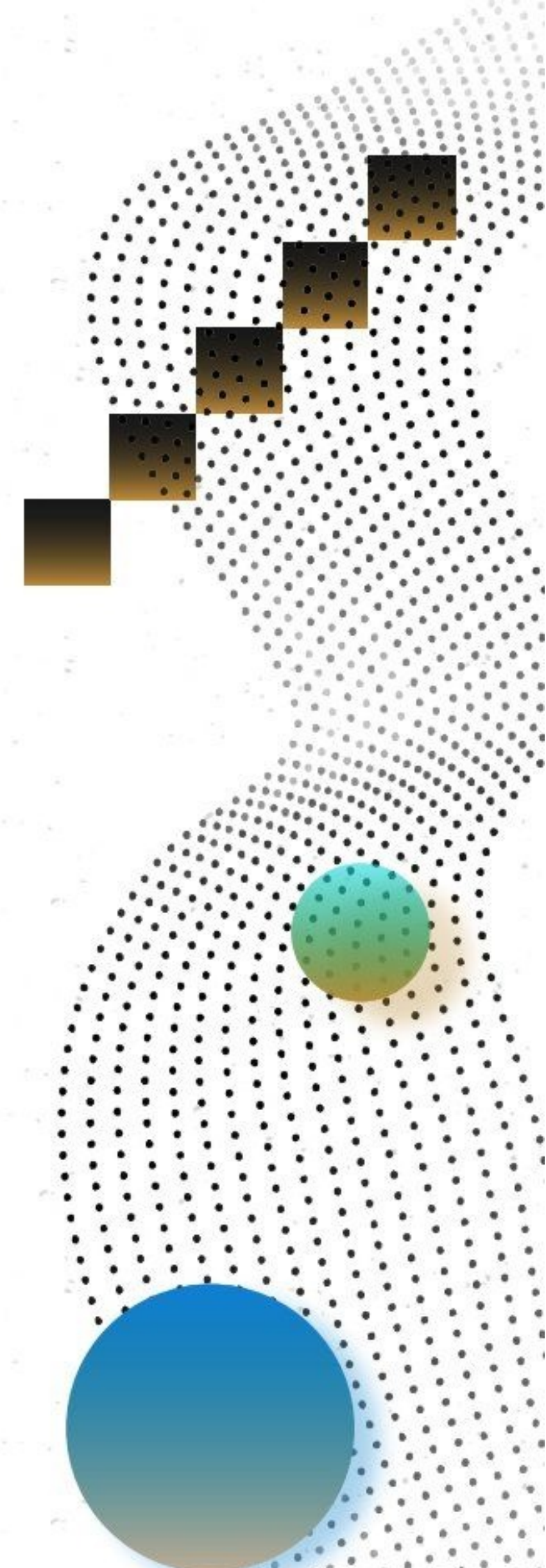
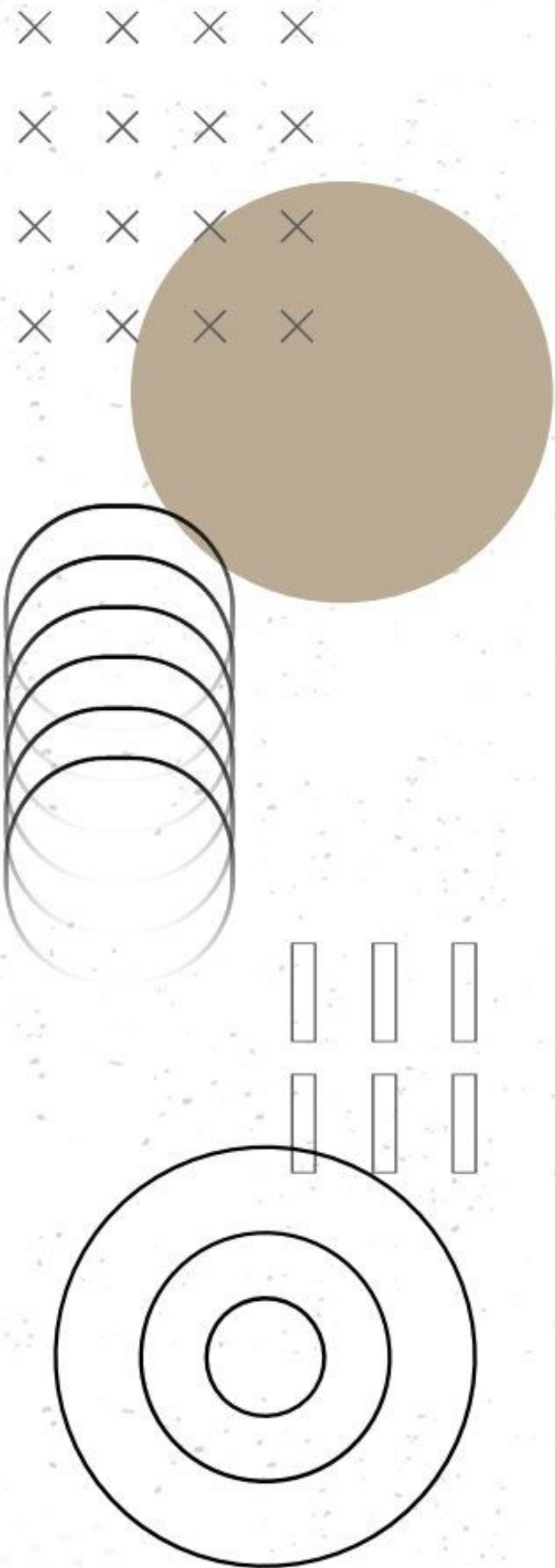


WHAT IS A DATA STRUCTURE

A DATA STRUCTURE IS A SPECIALIZED FORMAT FOR ORGANIZING, PROCESSING, RETRIEVING, AND STORING DATA EFFICIENTLY

ORGANIZE → STRUCTURE/ARRANGEMENT
STORE → MEMORY
EFFICIENTLY → FAST + LOW MEMORY

< T O >



THE PROBLEM WITHOUT STRUCTURE

SLOW SEARCHING
DIFFICULT UPDATES
MEMORY WASTE

< T O >

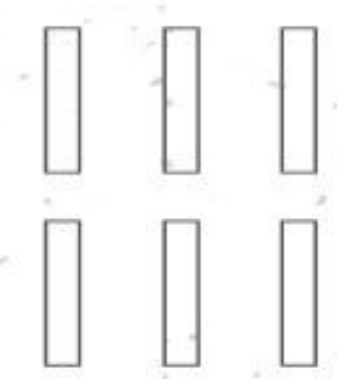


THE PROBLEM WITHOUT STRUCTURE

“LET’S SAY WE STORE ALL DATA RANDOMLY... NO ORDER, NO STRUCTURE.”

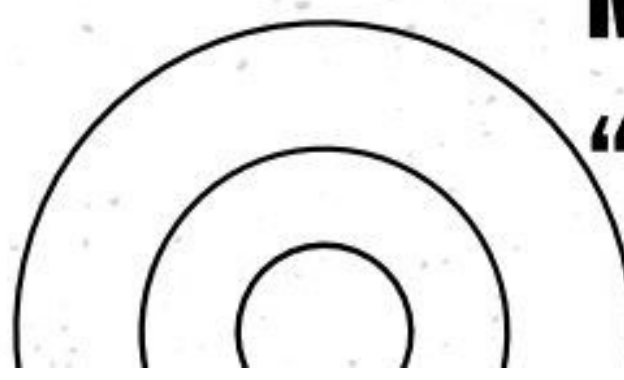
SLOW SEARCHING:

“YOU MIGHT NEED TO CHECK EVERYTHING ONE BY ONE.”



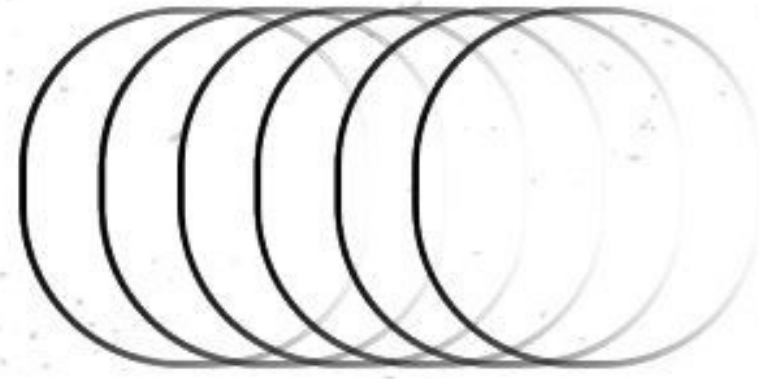
DIFFICULT UPDATES:

“INSERTING OR DELETING BECOMES MESSY.”



MEMORY WASTE:

“YOU MIGHT ALLOCATE SPACE POORLY.”



ORGANIZING DATA

**“WE ALWAYS HAVE DATA—NUMBERS, NAMES, IMAGES...
BUT THE QUESTION IS: HOW DO WE ARRANGE THIS DATA IN MEMORY?”**

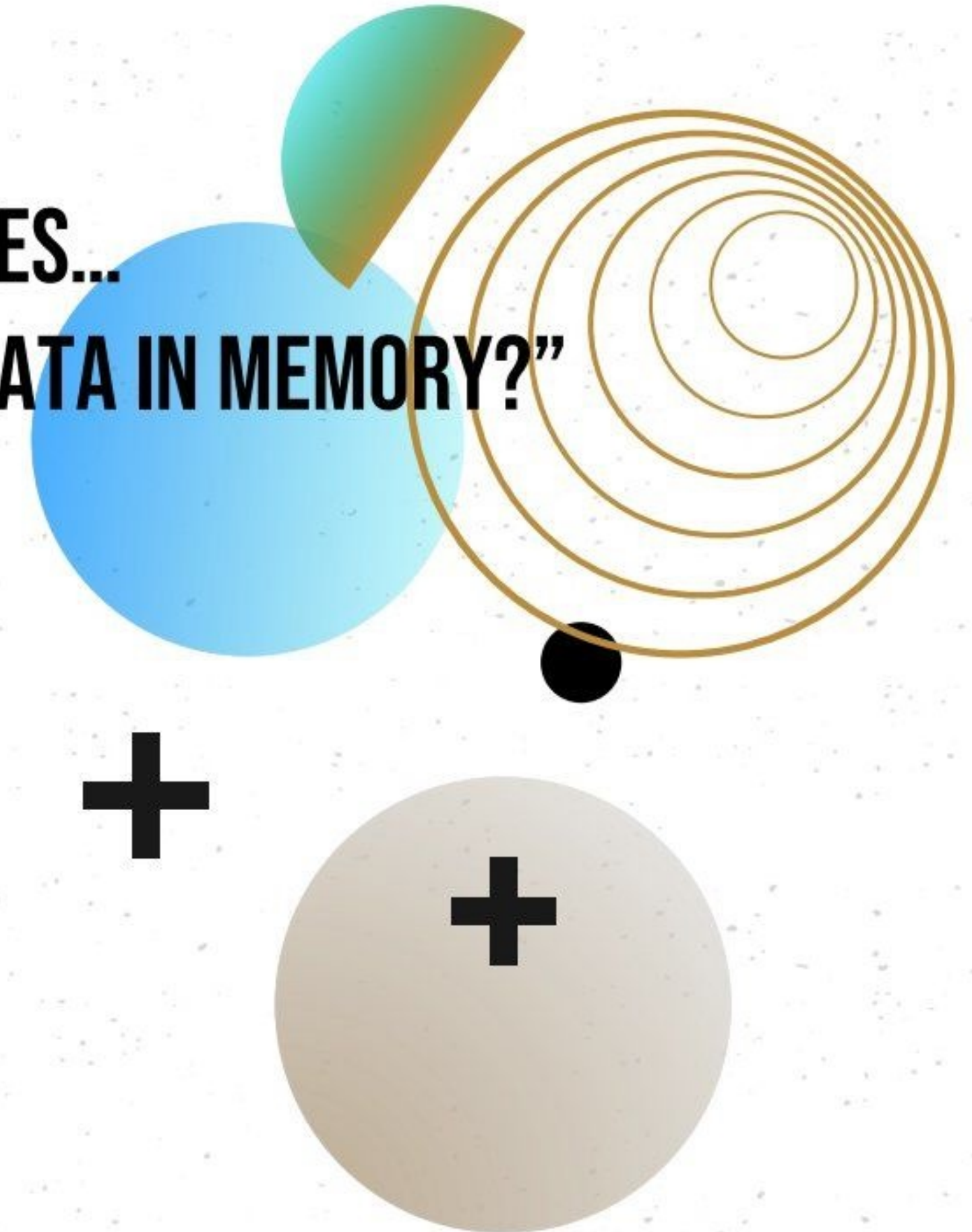
STACK → PLATES 🍴🍽️

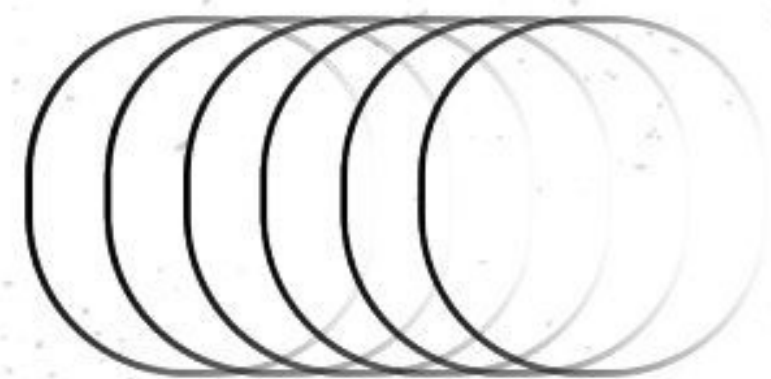
QUEUE → BANK LINE 🏦

ARRAY → LOCKERS 🗝️



< 0 1 >





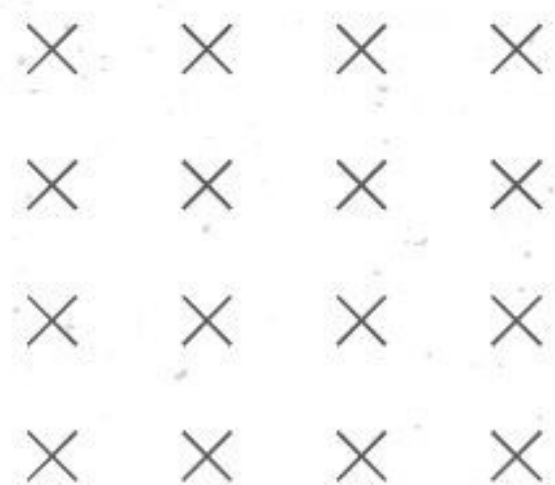
ORGANIZING DATA

STACK:

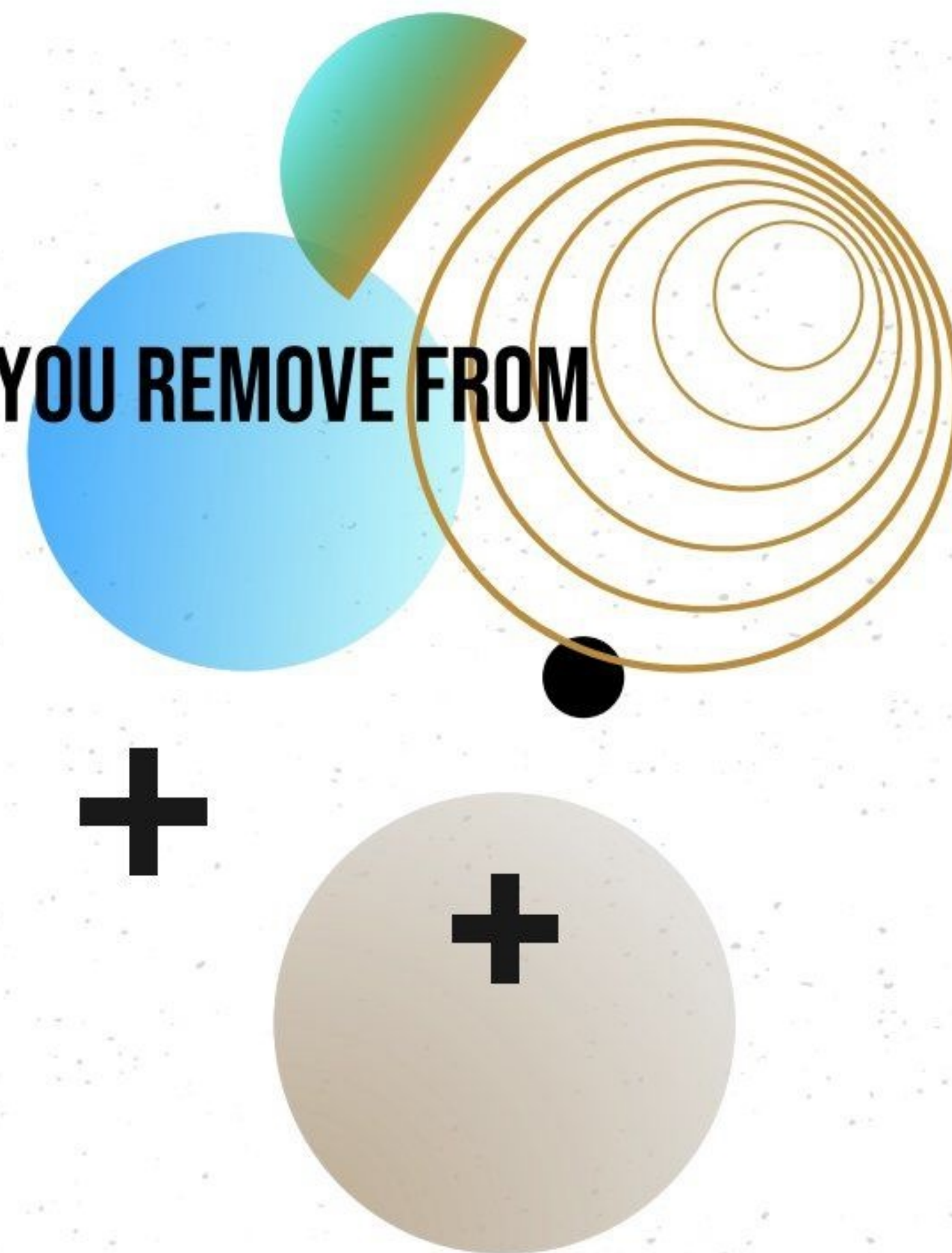
“IMAGINE PLATES. YOU ADD A PLATE ON TOP... AND YOU REMOVE FROM THE TOP.”

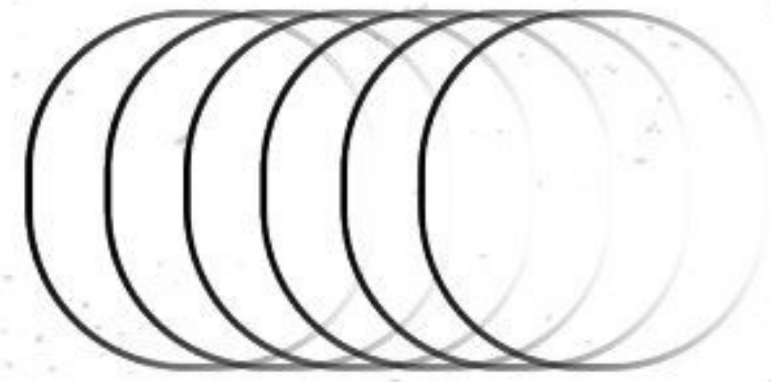
CONCEPT :

LAST IN → FIRST OUT (LIFO)



< 0 1 >





ORGANIZING DATA

QUEUE:

“IN A BANK LINE, THE FIRST PERSON COMES FIRST... AND LEAVES FIRST.”

CONCEPT:

FIRST IN → FIRST OUT (FIFO)

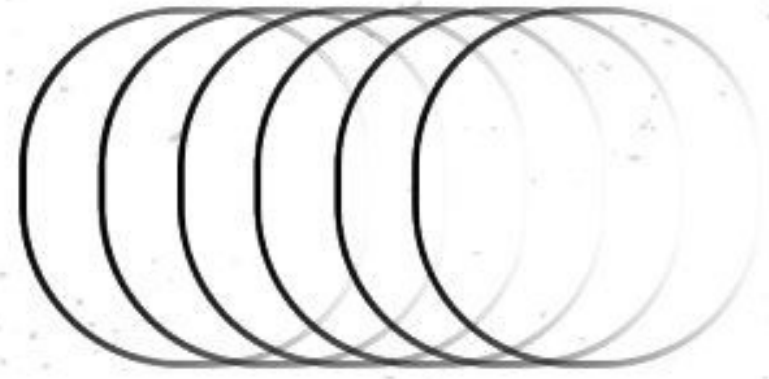


+

+



< 0 1 >



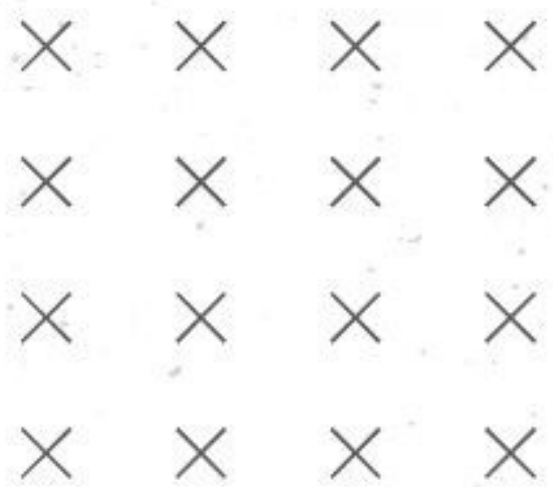
ORGANIZING DATA

ARRAY:

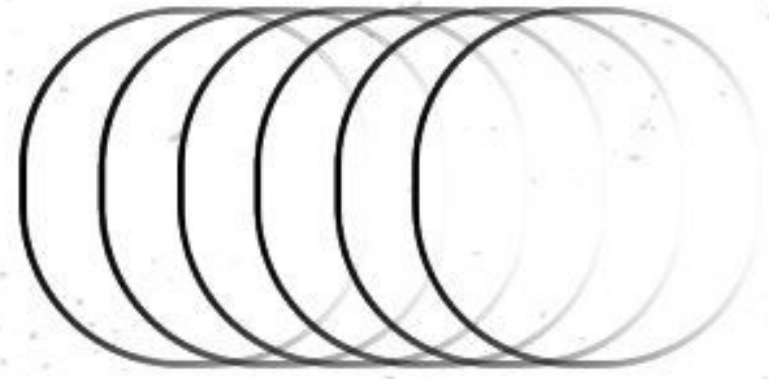
“LIKE LOCKERS—EACH POSITION HAS A FIXED PLACE.”

CONCEPT :

EACH ONE OF THE ELEMENTS HAS ITS OWN UNCHANGEABLE PLACE LIKE A PUZZLE PIECE.



< 0 1 >



CORE IDEA

SAME DATA + DIFFERENT STRUCTURE = DIFFERENT PERFORMANCE

EXAMPLE:

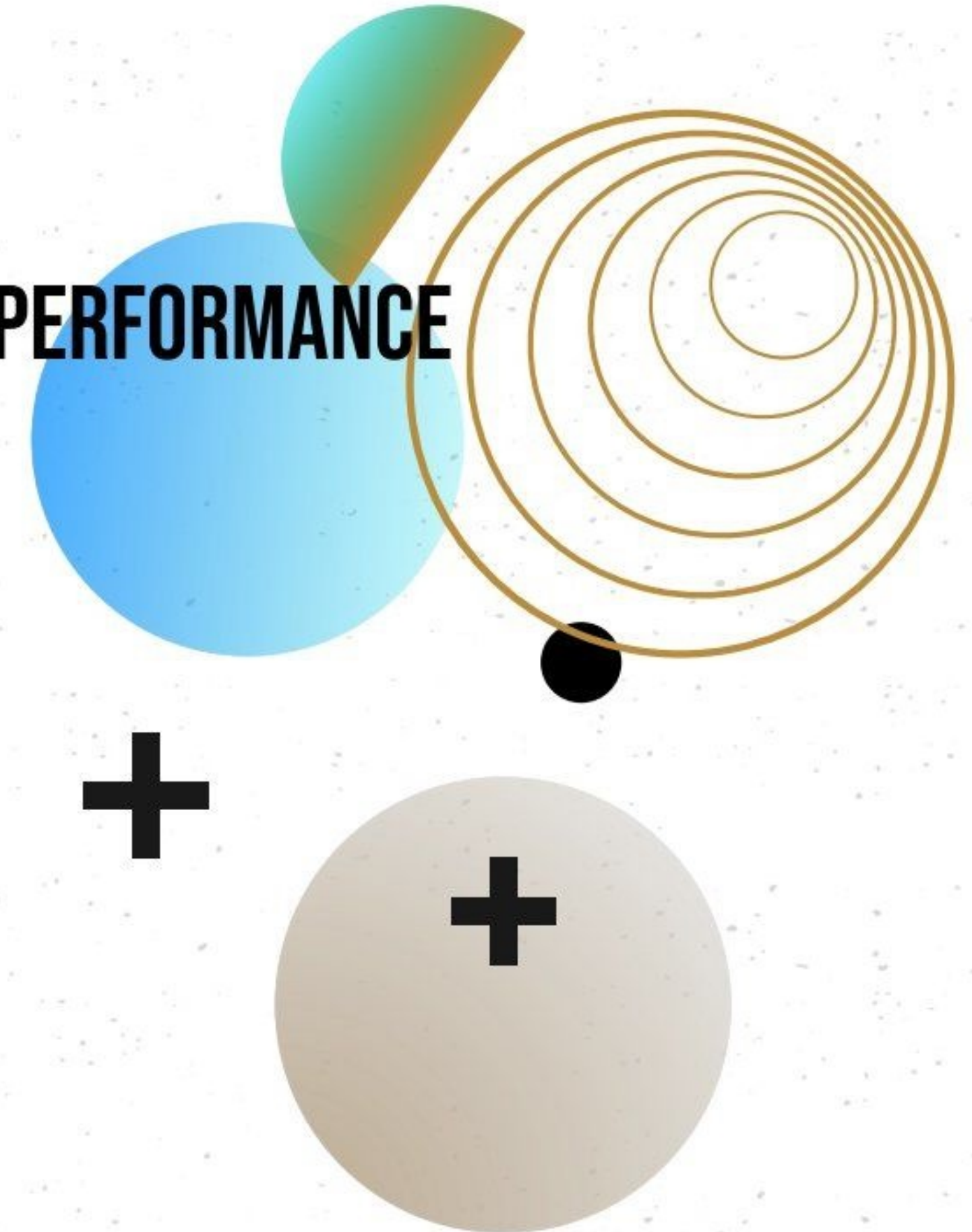
A LIST OF NAMES:

UNSORTED → SLOW SEARCH

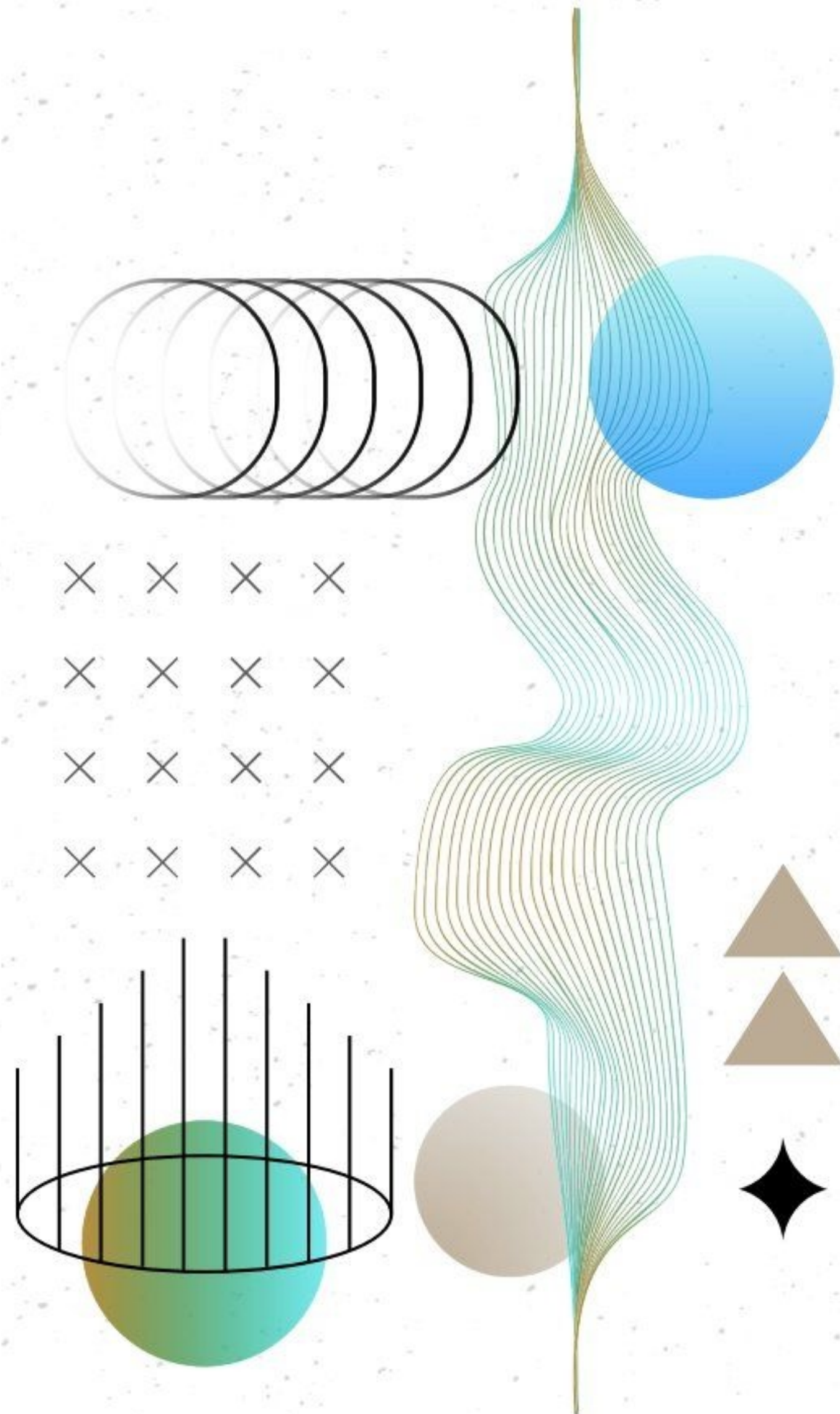
SORTED → FAST SEARCH



< 0 1 >



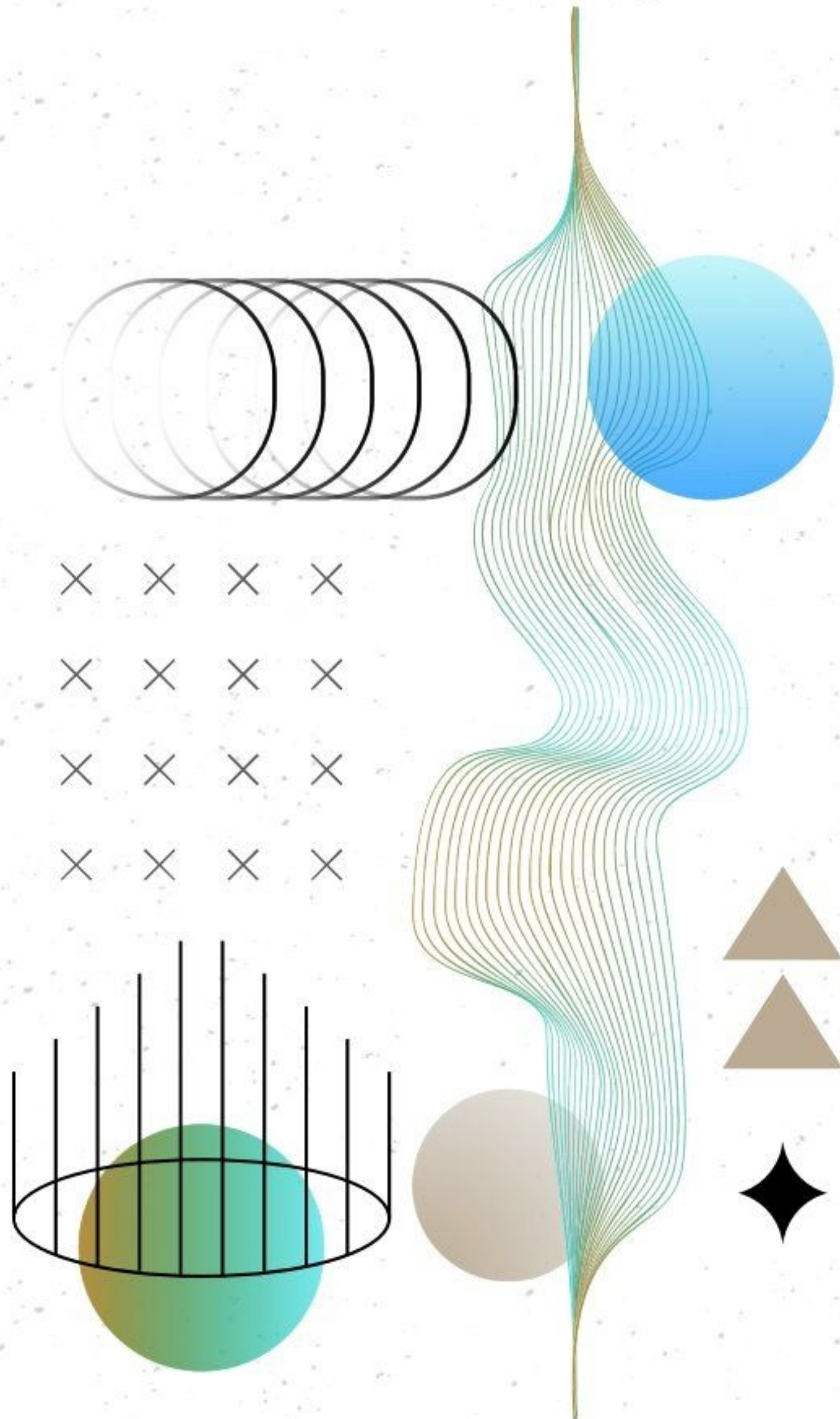
ABSTRACT DATA TYPE (ADT)



- An ADT is a model that defines what a data type does, but not how it is implemented.
- For example : remote buttons
- ADT = Concept
Data Structure = Implementation



ABSTRACT DATA TYPE (ADT)

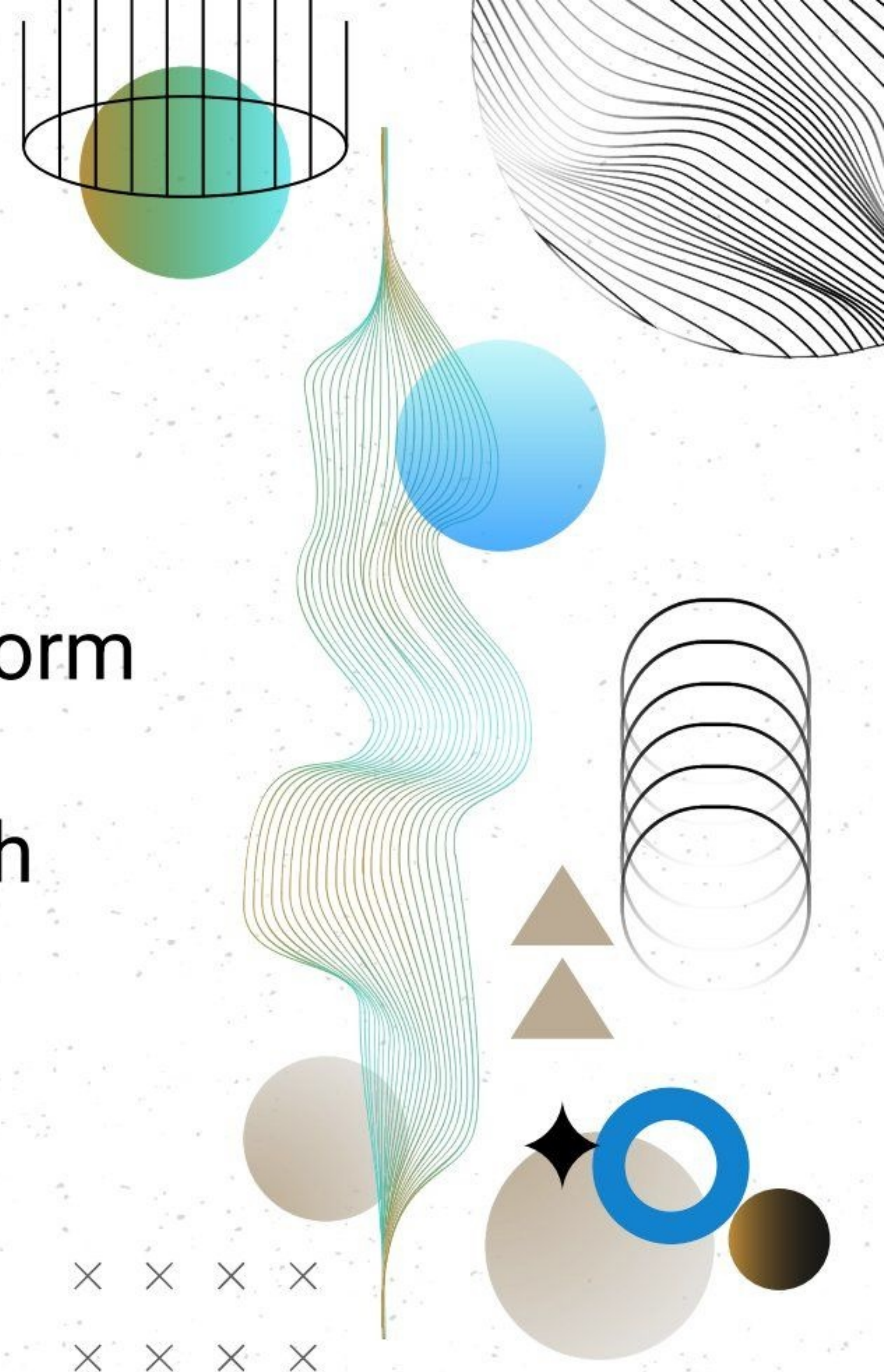


- “ADT is the idea...
Data Structure is how we build
that idea in memory.”



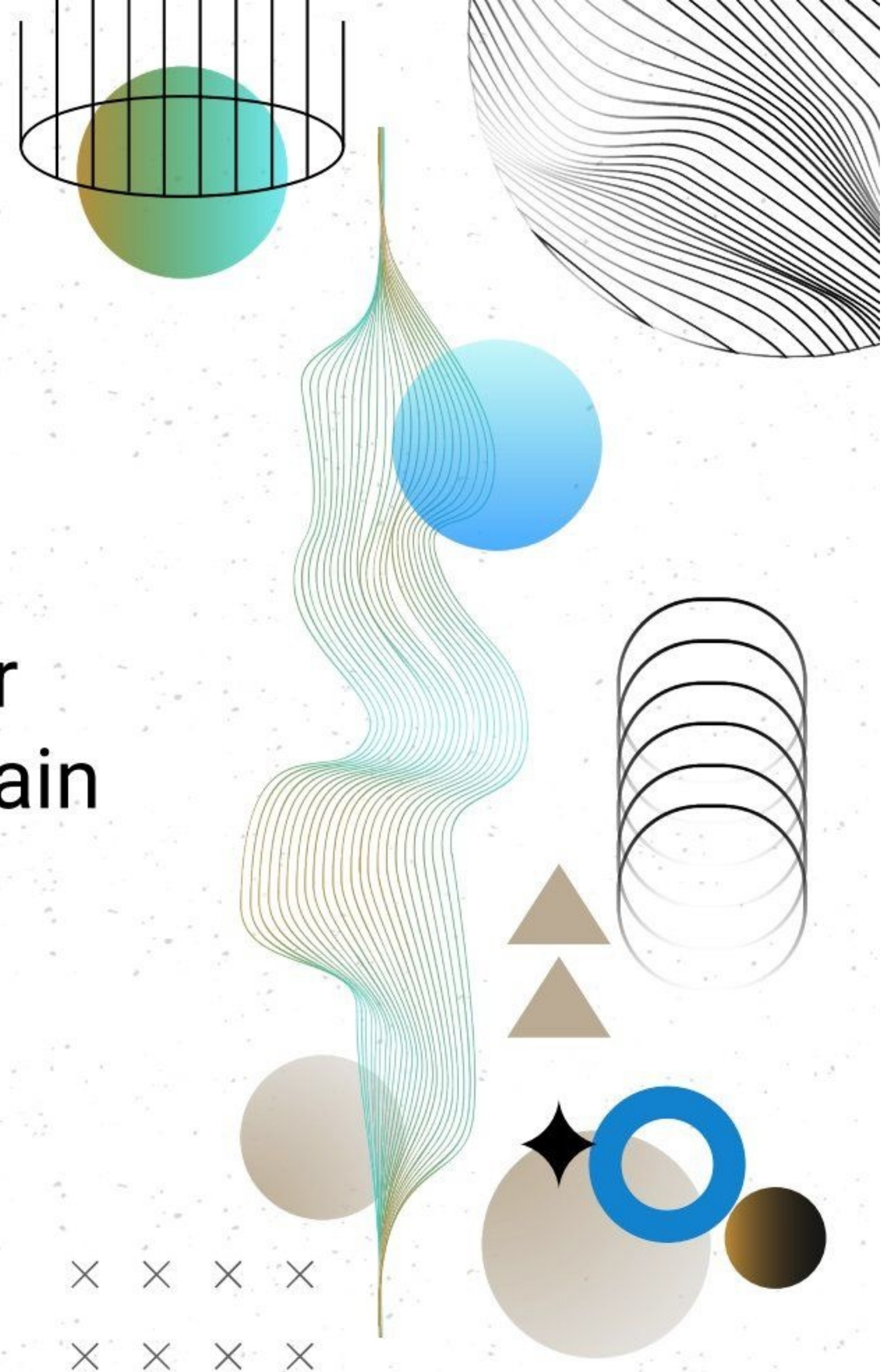
INTRODUCTION TO PERFORMANCE (COMPLEXITY)

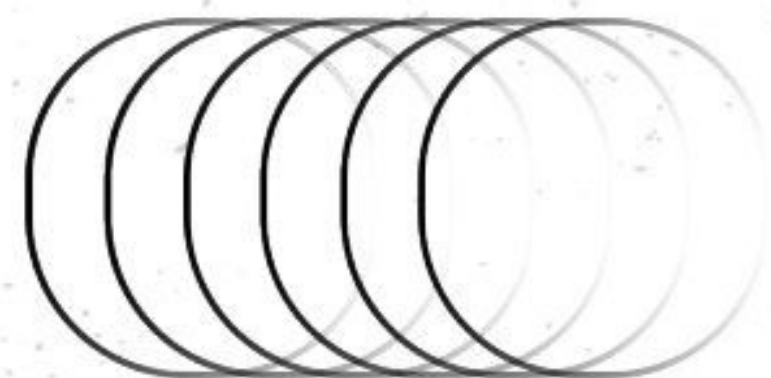
- Two Constraints:
 - 1. *Time Complexity:* How many operations does the algorithm perform as the data grows?
 - 2. *Space Complexity:* How much extra memory does the algorithm need?



INTRODUCTION TO PERFORMANCE (COMPLEXITY)

- The Reality:* We often trade one for the other (using more memory to gain more speed).





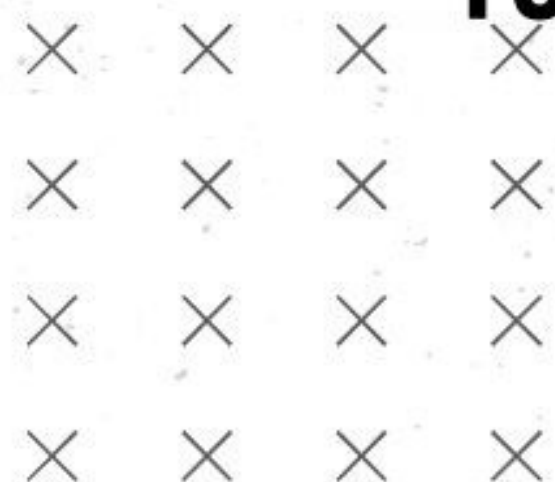
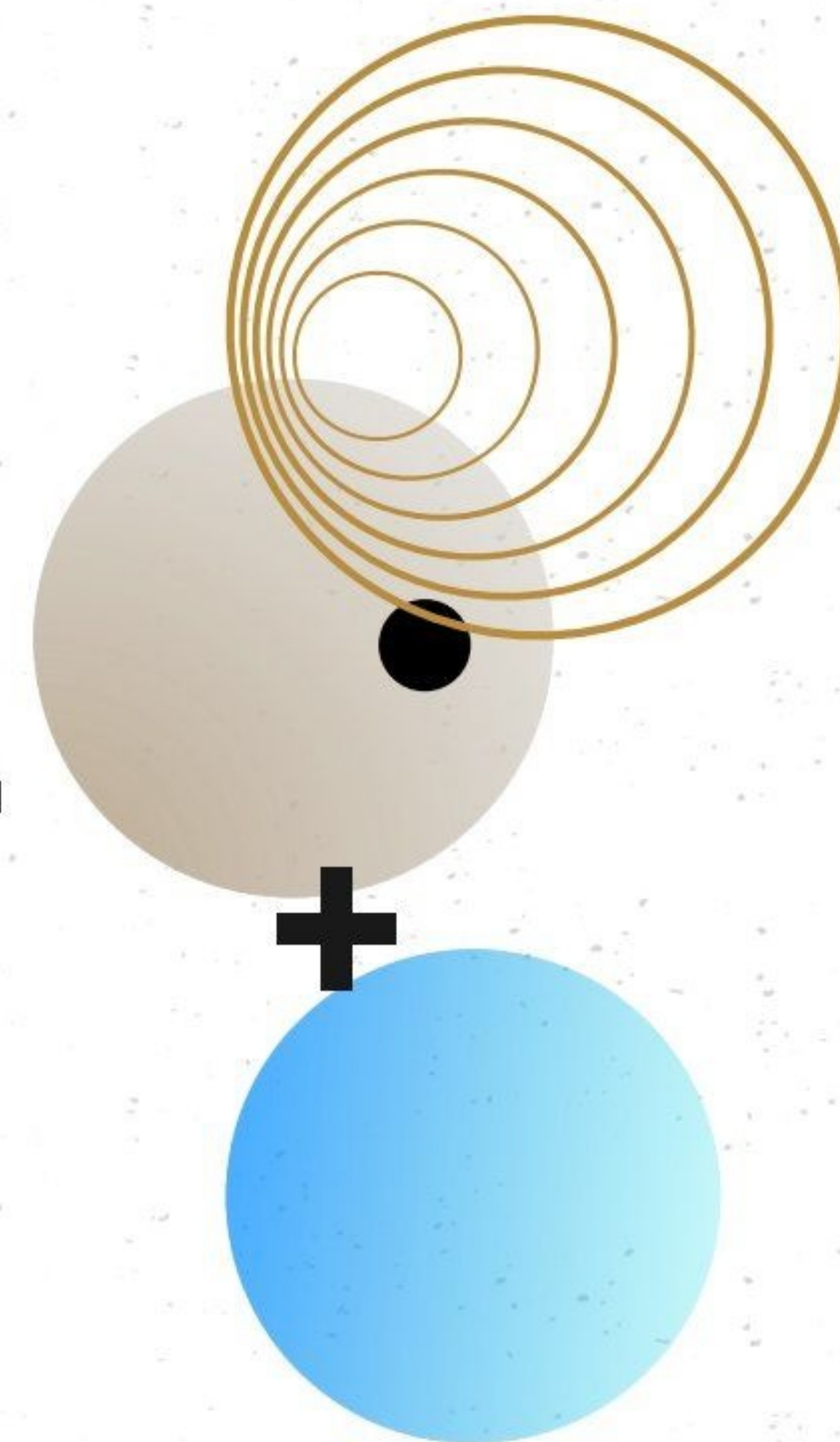
THE LANGUAGE OF EFFICIENCY

- BIG-O NOTATION

01 DEFINITION: * A MATHEMATICAL NOTATION USED TO DESCRIBE THE "WORST CASE SCENARIO" OF AN

02 ALGORITHM.

* *WHY "WORST CASE"?* IF YOU KNOW THE WORST IT
03 CAN PERFORM, YOU CAN GUARANTEE RELIABILITY IN **+**
YOUR SOFTWARE.



$\langle 0 \ 1 \rangle$

COMMON NOTATIONS

◆ $O(1)$ – CONSTANT

“NO MATTER HOW MUCH DATA WE HAVE... IT TAKES THE SAME TIME.”

EXAMPLE:

- ACCESSING ARRAY INDEX

$\langle \boxed{0} \boxed{1} \rangle$

COMMON NOTATIONS

◆ $O(N)$ — LINEAR

“TIME INCREASES AS DATA INCREASES.”

EXAMPLE:

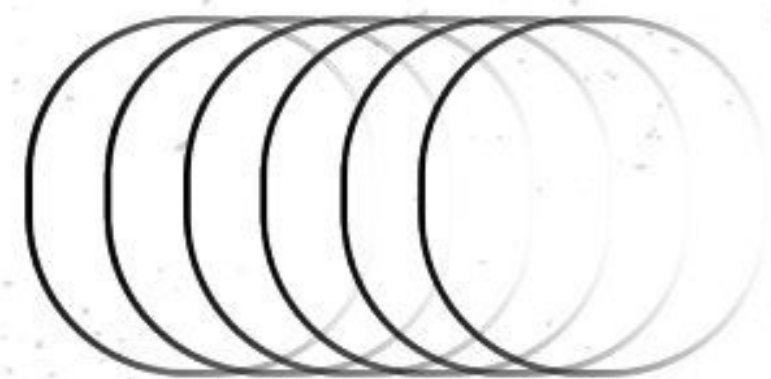
- SEARCHING ONE BY ONE

< 0 1 >

COMMON NOTATIONS

- ◆ $O(\log N)$ – LOGARITHMIC
“WE REDUCE THE PROBLEM SIZE STEP BY STEP.”
EXAMPLE:
 - BINARY SEARCH

< 0 1 >



NUMERICAL COMPARISON

N

10

1000

1,000,000

O(N)

10

1000

1,000,000

O(LOG N)

3

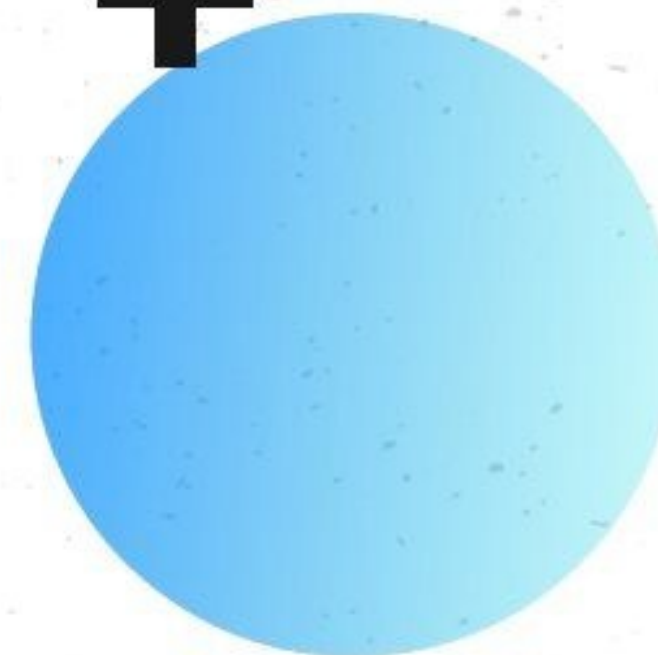
10

20~



+

+



× × × ×
× × × ×
× × × ×
× × × ×



< 0 1 >

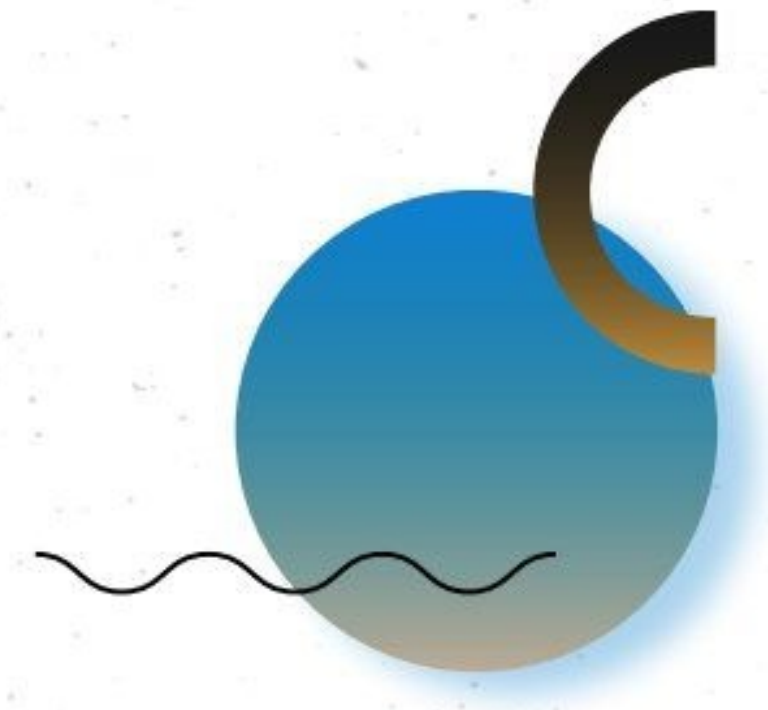
SUMMARY FOR YOU

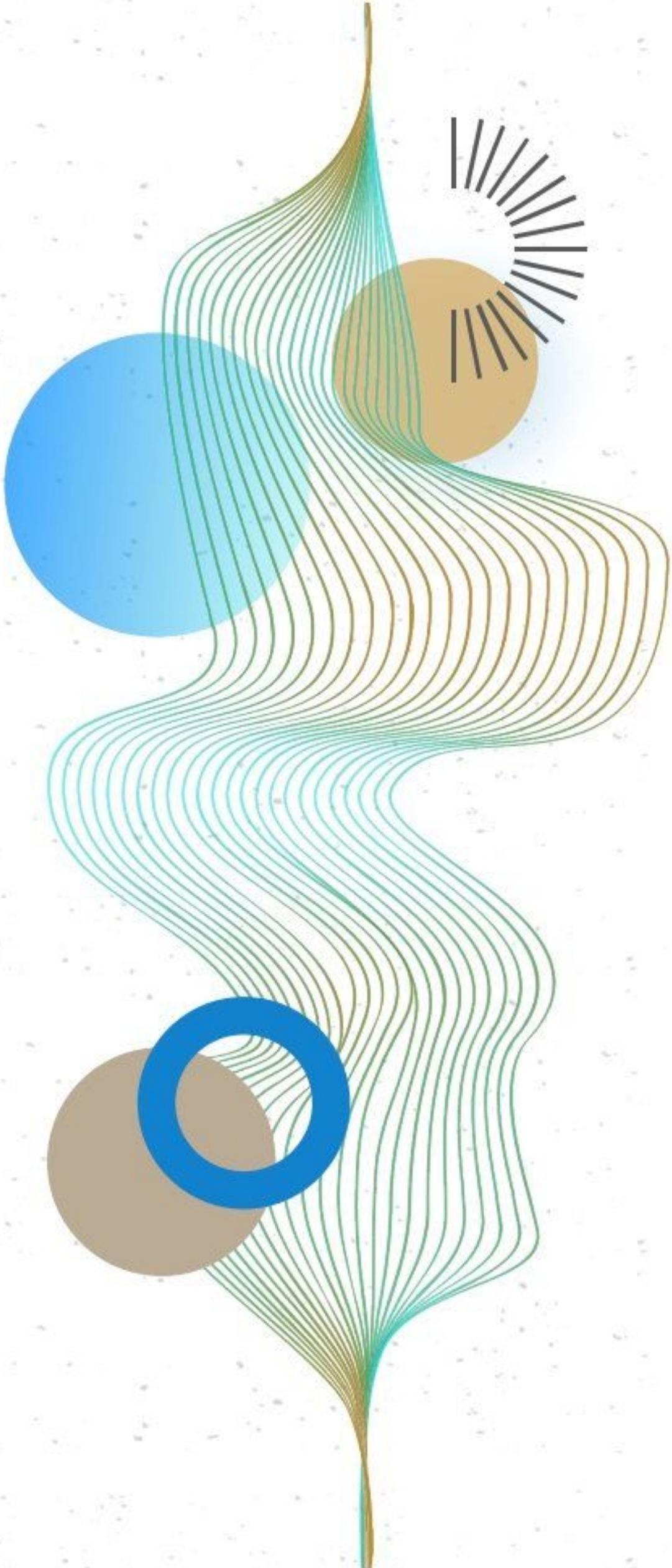
DATA STRUCTURES ORGANIZE DATA

ADT = CONCEPT

PERFORMANCE MATTERS

BIG-O MEASURES EFFICIENCY





THANK

YOU !

